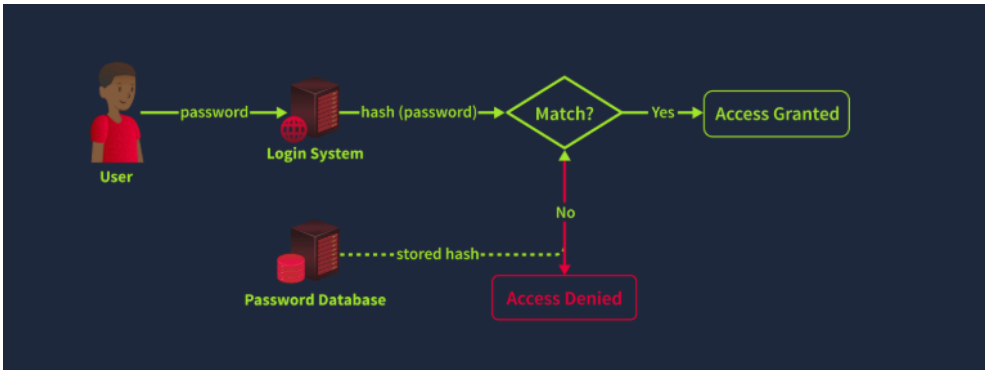
Jr.Pen 11.4.Password Cracking

Task 1 - Introduction

Passwords are the most common authentication mechanism on the internet and also one of the most reliable attack paths into a system. Whether we are dealing with a leaked database, a captured network handshake, or a hash extracted from a compromised machine, the ability to recover the plaintext from a hash is a fundamental red-team skill.

Task 2 - How Passwords Are Stored

Storing passwords in plain text leaves accounts instantly exposed in a breach, whereas storing only cryptographic hashes allows systems to verify logins securely without ever saving the original password.



Common Hashing Algorithms

Algorithm	Output Length	Still Used for Passwords?	Notes
MD5	128 bits (32 hex chars)	No	Fast, collision-prone, widely cracked
SHA-1	160 bits (40 hex chars)	No	Faster than SHA-256, deprecated
SHA-256	256 bits (64 hex chars)	Sometimes	Better than MD5/SHA-1 but still fast
NTLM	128 bits (32 hex chars)	Yes (legacy Windows authentication)	MD4-based, used for Windows account hashes
bcrypt	~ 60 chars, $2*$ prefix	Yes, recommended	Deliberately slow, cost-configurable
Argon2	Variable	Yes, recommended	Modern standard, memory-hard

Salting appends a unique random string to each password before hashing, ensuring identical passwords produce distinct hashes and rendering pre-computed rainbow tables useless.

- **One-way:** Hashing is irreversible; the original input can only be verified by hashing a candidate and comparing the outputs.
- **Deterministic:** Given the exact same input, a hash function will always produce the identical output.
- **Fixed-length output:** A hash algorithm consistently outputs a fixed number of characters regardless of the input size.
- **Collision-resistant:** It is computationally impractical for two distinct inputs to yield the same hash output; algorithms that fail this are deemed insecure.

Answer the questions below

> What property of a hash function means you cannot reverse a hash to recover the original input?

one-way

> What term describes the unique random string added to a password before hashing to prevent rainbow table attacks?

salt

> Which hashing algorithm was designed specifically for passwords and is deliberately slow by design?

bcrypt

Task 3 - Identifying Hash Types

The fastest check is always length and format:

Hash Type	Length	Prefix / Format	Example
MD5	32 hex chars	None	5f4dcc3b5aa765d61d8327deb882cf99
SHA-1	40 hex chars	None	5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
SHA-256	64 hex chars	None	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
NTLM	32 hex chars	None	8846f7eaee8fb117ad06bdd830b7586c
bcrypt	~60 chars	$2a$, $2b$, or $2y$	$2y$12$...

The trickiest ambiguity: **MD5 and NTLM are both 32 hex characters**. bcrypt hashes are impossible to confuse. The `$2a$`, `$2b$`, or `$2y$` prefix immediately identifies the algorithm.

Using Hashid: hashid analyses the format of a hash string and returns a list of possible algorithms.

```
root@tryhackme:~# hashid '5f4dcc3b5aa765d61d8327deb882cf99'
Analyzing '5f4dcc3b5aa765d61d8327deb882cf99'
```

For bcrypt, hashid gives an unambiguous result:

```
root@tryhackme:~# hashid '$2y$10$wJ/mZDURD4jQ0lrCEMheE.8FzMXNEBNjIkuZgEFm9VMn1m4ZP4eDG'
Analyzing '$2y$10$wJ/mZDURD4jQ0lrCEMheE.8FzMXNEBNjIkuZgEFm9VMn1m4ZP4eDG'
```

hashcat

```
root@tryhackme:~# hashcat --identify '5f4dcc3b5aa765d61d8327deb882cf99'
The following 11 hash-modes match the structure of your input hash:

      # | Name                                                       | Category
  ======+============================================================+=======
    900 | MD4                                                        | Raw Hash
      0 | MD5                                                        | Raw Hash
```

Online Resources:

- crackstation.net : Paste a hash, and it checks against a pre-computed lookup table of billions of entries. If the plaintext has been seen before, it comes back instantly.
- hashes.com : Identifies the hash type and attempts a lookup against a large database. Also accepts bulk hash submissions.

Algorithm	Hashcat Mode (-m)	John Format (--format=)
MD5	0	raw-md5
SHA-1	100	raw-sha1

Algorithm	Hashcat Mode (-m)	John Format (--format=)
SHA-256	1400	raw-sha256
SHA-512	1700	raw-sha512
NTLM	1000	nt
bcrypt	3200	bcrypt

Answer the questions below

> What is the character length of an SHA-256 hash?

64

> What John the Ripper format would you use for an SHA-256 hash?

raw-sha256

Task 4 - Wordlists and Attack Strategies

Hashcat rules at `/opt/hashcat/rules/`, and John rules at `/opt/john/rules/`. If a path does not exist, use `find / -name <filename> 2>/dev/null` to locate it. The most widely used wordlist is **rockyou.txt**, a real collection of 14 million passwords leaked in the 2009 RockYou breach. It is pre-installed on the AttackBox at `/usr/share/wordlists/rockyou.txt`. For broader coverage, the **SecLists** collection at `/usr/share/wordlists/SecLists/` includes targeted wordlists for specific contexts: common web application passwords, application-specific defaults, and country-specific lists. The path `/usr/share/wordlists/SecLists/Passwords/` is the one we use most often.

Dictionary attacks: A dictionary attack tests a pre-built list of candidate passwords against the hash, one by one. It is the fastest first step for most cracking tasks and should always be our starting point.

Brute-Force Attacks: A brute-force attack generates every possible character combination up to a specified length. Given unlimited time, it will always recover the password. In practice, it is rarely viable beyond 6-7 characters because the search space grows exponentially.

Rule-Based Attacks: Rule-based attacks take an existing wordlist and apply transformations to each word, generating the mutations that people commonly use when constructing passwords.

Rule File	Description
best64.rule	64 highly effective mutations, good first choice
rockyou-30000.rule	30,000 rules derived from RockYou analysis
d3ad0ne.rule	Large community-built rule set
dive.rule	An extensive rule set covering a wide range of mutations
OneRuleToRuleThemAll.rule	Popular community-compiled rule set; not bundled by default, verify it exists on the system before use

Mask Attacks: A mask attack is a structured Brute-force attack in which you define the **pattern** of the password rather than just the character set. If we know a password follows a specific structure, for example, a word followed by a four-digit year, a mask attack generates only candidates that match that structure.

Hashcat mask syntax uses placeholders for character sets:

Placeholder	Character Set
?l	Lowercase letters (a-z)
?u	Uppercase letters (A-Z)
?d	Digits (0-9)
?s	Special characters

Placeholder	Character Set
?a	All printable ASCII

Scenario	Best approach
No information about the password	Dictionary attack with rockyou.txt
Dictionary fails, password likely mutated	Dictionary + rules (e.g., best64.rule)
Known password pattern or enforced policy	Mask attack
Short password, small character set	Brute force (constrained length only)
Target likely used company-specific terms	Custom wordlist + rules

Answer the questions below

> What is the full path to the rockyou wordlist on the AttackBox?

/usr/share/wordlists/rockyou.txt

> What type of attack applies transformations such as capitalisation and number suffixes to an existing wordlist?

rule-based

> A password policy requires all passwords to start with a capital letter followed by six lowercase letters and two digits. What attack type is best suited to cracking a hash of such a password?

mask

Task 5 - Cracking with John the Ripper and Hashcat

John the Ripper and **Hashcat** are the two tools we will use most for offline cracking. Both handle dictionary attacks, rules, and masks. Where they differ is in speed, format support, and how they handle output.

All examples in this task use two MD5 hashes. Write them to files before running anything:

```
echo "5f4dcc3b5aa765d61d8327deb882cf99" > demo.txt
echo "0571749e2ac330a7455809c6b0e7af90" > demo2.txt
echo "37b4e2d82900d5e94b8da524fbeb33c0" > demo3.txt
```

`demo.txt` is used for basic dictionary attacks, `demo2.txt` for rule-based attacks, and `demo3.txt` for the mask attack.

John the Ripper: John the Ripper is a versatile CPU-based cracker. It handles a wide range of hash formats, including many non-standard ones, and can auto-detect formats when we do not specify one explicitly. It is particularly good for Unix shadow file entries and for quick attempts that require automatic format detection.

```
root@tryhackme:~# john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt demo.
txt
Created directory: /root/.john
```

- `-format=raw-md5`, specifies the hash format explicitly (use the John format labels from Task 3)
- `-wordlist=`, path to the wordlist file
- `demo.txt`, the file containing one hash per line

Auto-detect mode (when unsure of the format):

```
root@tryhackme:~# john --wordlist=/usr/share/wordlists/rockyou.txt demo.txt
```

john --rules

```
root@tryhackme:~# john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt --rul
es=wordlist demo2.txt
Using default input encoding: UTF-8
```

Viewing Cracked Passwords: John stores results in its potfile at `/usr/local/john/run/john.pot`. To display previously cracked hashes: john --show

```
root@tryhackme:~# john --show --format=raw-md5 demo.txt
?:password
```

Hashcat: Hashcat is GPU-accelerated, making it significantly faster than John for most hash types. On a modern GPU, Hashcat can process billions of MD5 or SHA-1 candidates per second, compared to millions on the CPU. For bcrypt, the speed difference matters less because bcrypt's cost factor limits throughput regardless of hardware.

```
root@tryhackme:~# hashcat -m 0 -a 0 demo.txt /usr/share/wordlists/rockyou.txt

hashcat (v6.2.6) starting
...

5f4dcc3b5aa765d61d8327deb882cf99:password
```

Flags explained:

- `m 0`, hash type (0 = MD5; use the mode numbers from Task 3)
- `a 0`, attack mode (0 = dictionary)
- `demo.txt`, the file containing one hash per line
- The final argument is the wordlist path

Rule-Based Attack:

```
root@tryhackme:~# hashcat -m 0 -a 0 demo2.txt /usr/share/wordlists/rockyou.txt -r /usr/lo
cal/hashcat/rules/best64.rule
hashcat (v6.2.6) starting

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 1104517645
```

	John the Ripper	Hashcat
Acceleration	CPU (primarily)	GPU (primarily, CPU fallback)
Speed (MD5/SHA)	Fast	Very fast
Format detection	Good auto-detect	Explicit mode required
Non-standard formats	Excellent	Good
Rule sets	Built-in + extensible	Large file library
Best for	Quick attempts, varied formats, shadow files	Sustained attacks, GPU-accelerated cracking

Answer the questions below

> A penetration tester needs to crack a large set of SHA-256 hashes as quickly as possible. Which tool is the better choice: John the Ripper or Hashcat?

hashcat

> What Hashcat attack mode number is used for a mask attack?

3

> Which John the Ripper flag displays previously cracked passwords from the potfile?

--show

Task 6 - Practical

he four hash files are pre-loaded on the AttackBox in `/root/Rooms/PasswordCracking/`. If working on a local machine, download the zip by clicking the **Download Task Files** button and extract it to get `hash1.txt`, `hash2.txt`, `hash3.txt`, and `hash4.txt`. Work through the files in order. For each one:

Work through the hashes in order. For each one:

1. Run `hashid` on it and cross-reference the output with the visual characteristics from Task 3 to confirm the algorithm
2. Select the appropriate Hashcat mode or John format from Task 3's reference table
3. Run a dictionary attack using `/usr/share/wordlists/rockyou.txt`

Hash 4 is bcrypt. It will crack more slowly than the others because bcrypt is designed to be expensive. Be patient.

Two of the four hashes have the same plaintext. That is deliberate. If we run them side by side, the outputs look nothing alike, which is exactly the point: the algorithm matters as much as the password itself. Identifying the hash type is not a formality.

Answer the questions below

> What algorithm produced the hash in `hash1.txt`?

md5



> What Hashcat mode would you use to crack the hash in `hash2.txt`?

1400 = sha256

> What is the plaintext password for `hash1.txt`?

123456

> `hash2.txt` and `hash3.txt` have different algorithms but share the same plaintext. What is it?

password

> What is the plaintext password for `hash4.txt`?

Task 7 - Conclusion

The gap between "I have a hash" and "I have a password" is shorter than it looks. Identify the algorithm, pick an attack that fits what we know about the target, and point the right tool at it. The bcrypt hash in Task 6 should have taken noticeably longer than the others. That is not a bug, it is the point of a slow hash function working exactly as intended.

We moved from how passwords are stored and why algorithm choice matters, through hash identification with `hashid` and `hashcat --identify`, to running dictionary, rule-based, and mask attacks with both John the Ripper and Hashcat. The same knowledge works in reverse: choosing bcrypt or Argon2 with a high cost factor, enforcing salting, and avoiding fast hashes for password storage are the defensive side of everything we practised here.